
Handling CORS & UI Polish (Theory)

Welcome to Handling CORS & UI Polish! As you prepare to connect your React frontend to your FastAPI backend in upcoming modules, you will inevitably encounter browser security barriers. In this session, you will learn about **CORS (Cross-Origin Resource Sharing)** and best practices for polishing your ML prediction user interface.

1. Understanding CORS (Cross-Origin Resource Sharing)

By default, modern web browsers enforce a security measure called the **Same-Origin Policy**. This policy prevents a webpage running on one origin (e.g., your Vite development server at `http://localhost:5173`) from making requests to a server running on a *different* origin (e.g., your FastAPI server at `http://localhost:8000`), unless the server explicitly allows it.

A. What is an Origin?

An origin is defined by the combination of:

- **Protocol:** (e.g., `http://` vs `https://`)
- **Domain/Host:** (e.g., `localhost` or `example.com`)
- **Port:** (e.g., `5173` vs `8000`)

Even though both your React app and FastAPI app run on your local machine, **different ports mean different origins**.

B. How CORS Works

1. Your browser sends a preflight request (`OPTIONS`) to the FastAPI server asking: "*Am I allowed to send data here?*"
 2. Your FastAPI backend responds with CORS headers specifying which frontend origins, methods (GET, POST), and headers are permitted.
 3. If approved, the browser lets the actual request go through. If not, it blocks the request and throws a CORS policy error in the console.
-

2. UI Polish & UX Best Practices for ML Dashboards

When building applications that interact with machine learning models, user experience (UX) is critical because model inference can sometimes take a fraction of a second or require clear visual feedback.

A. Visual Feedback States

- **Idle State:** Clean form ready for input with clear placeholder examples.
- **Loading State:** Disabled submit buttons and animated spinners/text to prevent double-submissions while the backend processes inference.
- **Success State:** Highlighted prediction results with readable typography and confidence metrics.
- **Error State:** Distinct warning alerts if the server is offline or validation fails.

B. Input Validation

Before sending data across the network, ensure client-side validation catches empty fields, negative numbers (where invalid, like square footage), or incorrect data types to minimize unnecessary server load.

3. Reference Links & Further Reading

- **MDN CORS Overview:** Deep dive into how cross-origin resource sharing works in browsers.
 - **FastAPI CORS Middleware Documentation:** Official guide on how to configure CORS middleware in your Python backend.
-